

Over-Squashing vs. Adversarial Robustness in GNNs: An “Add vs. Remove Edge” Investigation

Supanut Kompayak (st126055) Dechathon Niamsa-Ard (st126235)
Deep Learning Course Project, Asian Institute of Technology
{st126055, st126235}@ait.asia

Midway Report — June 28, 2026

Abstract

Graph neural networks (GNNs) suffer from two seemingly unrelated structural problems: *over-squashing*, where long-range information is compressed through bottlenecks and lost, and *adversarial fragility*, where a few maliciously edited edges flip predictions. Both concern how sensitive a node’s output is to structural change elsewhere. As a learning project, we *investigate* whether methods that mitigate over-squashing also help adversarial robustness, through the lens of **adding vs. removing edges**: over-squashing rewiring methods either *add* edges (FoSR) or *remove* them (spectral pruning), while adversarial attacks add edges and adversarial defenses (GCN-Jaccard) remove them. On Cora node classification we compare a vanilla GCN, an add-based rewiring (FoSR), a remove-based spectral pruning (Jamadandi ProxyDelete), and a purpose-built defense (GCN-Jaccard) under the Metattack poisoning attack, reporting clean accuracy, robust accuracy, and the robustness gap, plus a Jacobian-based over-squashing probe. We report only real, reproducible numbers; negative results are acceptable. Preliminary work establishes a GCN/Cora baseline of $80.30 \pm 0.47\%$ and validates the full attack–defense pipeline end-to-end.

1 Introduction

Message-passing GNNs propagate information hop-by-hop along edges. Two known failure modes both stem from how information flows through graph structure. **Over-squashing** [1, 8, 2]: reaching distant nodes requires many layers, but the exponentially growing long-range signal is squeezed through structural bottlenecks into a fixed-size vector and lost — the output is *not sensitive enough* to distant signal. **Adversarial fragility** [10, 11]: an attacker adds or removes a few edges and message passing pulls in the bad signal, flipping the prediction — the output is *too sensitive* to malicious structural change. Both are about the sensitivity of a node’s output to structural change elsewhere; this conceptual bridge is what we investigate, not a claim we prove.

The sharper framing we use is **add vs. remove edge**. Over-squashing rewiring mostly *adds* edges to raise the spectral gap and relieve bottlenecks (FoSR [5], SDRF [8]); adversarial attacks typically *add* edges between dissimilar nodes; adversarial defenses *remove* suspicious edges (GCN-Jaccard [9], Pro-GNN [4]). Thus adding edges to fix over-squashing might *hurt* robustness (more edges to exploit), while a *remove*-based over-squashing method — spectral pruning [3] — might transfer to robustness because it removes edges like a defense does. “Add vs. remove: which helps robustness, and why?” is the spine of this project.

2 Research Questions

- **RQ2 (lead).** Do over-squashing mitigations (add-based FoSR vs. remove-based spectral pruning) improve adversarial robustness relative to (a) an undefended GCN and (b) a purpose-built defense (GCN-Jaccard), and at what cost to clean accuracy?
- **RQ1 (secondary).** Does structural perturbation measurably worsen an over-squashing measure? Is there a relationship between perturbation strength and long-range information loss?
- **RQ3 (stretch).** Can a single measure (Jacobian sensitivity) capture both over-squashing severity and adversarial fragility on the same scale?

3 Data

Cora is the workhorse; everything else is optional. All statistics are confirmed at load time.

Dataset	Task	Size	Role
Cora (Planetoid)	node clf.	2,708 nodes / 5,278 undir. edges / 7 cls / 1,433 feat	primary
Citeseer (Planetoid)	node clf.	3,327 / 4,552 / 6 / 3,703	2nd setting
Peptides-func (LRGB)	graph clf. (AP)	15,535 graphs, ~150 avg nodes	stretch
ZINC (subset)	graph reg. (MAE)	12,000 graphs, ~23 avg nodes	stretch

Table 1: Datasets. Cora is the primary node-level and standard adversarial-attack testbed. Long-range graph-classification sets (LRGB, ZINC) are stretch goals for the over-squashing probe.

4 Methods and Approach

Backbone. GCN [6] is the primary backbone (2 layers, hidden 16, dropout 0.5). GAT and Graph-SAGE are stretch.

The method table (the design). Each method occupies a position on the add/remove axis:

Method	Targets	Edge op	Role
GCN (vanilla)	—	none	baseline
FoSR	over-squashing	add	the “add” arm
Spectral pruning (ProxyDelete)	over-squashing	remove	the “remove” arm
GCN-Jaccard	adversarial defense	remove	purpose-built defense reference
DropEdge	regularization	remove (random)	random-removal sanity baseline

Table 2: What each comparison teaches: *FoSR* vs. *ProxyDelete* isolates the add/remove *direction* among over-squashing fixes; *ProxyDelete* vs. *GCN-Jaccard* asks whether spectral-criterion removal matches similarity-criterion removal; *all* vs. *vanilla* asks whether any of this helps robustness, and at what clean-accuracy cost.

Attack and defense. The attack is Metattack [10], a global meta-learning poisoning attack and the standard testbed for robust-accuracy benchmarks, at perturbation rates $\{0.05, 0.10\}$ (with 0.20 as stretch). The defense reference is GCN-Jaccard [9], which prunes edges between dissimilar nodes. We use DeepRobust [7] for both. The remove-based over-squashing arm is the ProxyDelete spectral pruning of Jamadandi et al. [3], which deletes edges to maximize the spectral gap; the add-based arm is FoSR [5], adapted from graph- to node-classification.

Pipeline order (the operational spine). Metattack is a *poisoning* attack, so the order of operations is part of the design, not an implementation detail:

$$\text{clean graph} \xrightarrow{\text{Metattack}} \text{poisoned } G' \xrightarrow{M} G'' \xrightarrow{\text{train GCN}} \text{eval}, \quad M \in \{\text{none, FoSR, ProxyDelete, GCN-Jaccard}\}.$$

The attack precedes training by definition of poisoning; the method M is applied to the received (possibly poisoned) graph before training, mirroring deployment and matching how DeepRobust evaluates GCN-Jaccard. Because a method changes the graph even with no attack, its clean-accuracy cost must be measured separately, giving four graph conditions per method: clean; clean+ M ; poisoned; and poisoned+ M . The headline table reports, per method, clean accuracy, robust accuracy at ptb $\{0.05, 0.10\}$, and the robustness gap (clean – robust).

Metrics. (i) *Clean accuracy*; (ii) *robust accuracy* under Metattack; (iii) *robustness gap* = clean – robust (headline); and (iv) an *over-squashing measure* based on Jacobian sensitivity $\partial \mathbf{h}_v / \partial \mathbf{x}_u$ for distant node pairs (u, v) [2], computed via autograd (scoped to a few far pairs first, then expanded). The focus of the investigation is RQ2; RQ1 follows almost for free once the probe exists.

5 Discussion

The grid directly answers RQ2. The attacker mostly *adds* dissimilar edges; GCN-Jaccard *removes* exactly those (which is why removal defends); FoSR *adds* more edges (does adding hurt robustness?); ProxyDelete *removes* edges by a spectral criterion (does over-squashing-motivated removal happen to remove attacker edges too?). *FoSR vs. ProxyDelete* isolates the effect of edge direction among methods with the same over-squashing goal, while *ProxyDelete vs. GCN-Jaccard* isolates whether the removal *criterion* matters — i.e. whether generic spectral removal suffices or removal must target the attack. Our working hypothesis is that add-based rewiring is neutral-to-harmful for robustness while remove-based pruning partially transfers, but a negative or weak result is an acceptable and informative outcome. The Jacobian probe links the two phenomena: if perturbation increases long-range sensitivity (RQ1) and the same measure tracks fragility (RQ3), the conceptual bridge is empirically supported on one scale.

Caveat. We use a non-adaptive attacker: Metattack attacks a vanilla GCN surrogate and is unaware of the defense applied afterward (the DeepRobust standard). This flatters defenses relative to an adaptive attacker; we note it as a limitation and treat adaptive attacks as a stretch goal.

6 Summary of Progress

Done. (1) A reproducible GCN/Cora baseline of $80.30 \pm 0.47\%$ test accuracy (5 seeds, CPU), confirming Cora statistics at load. (2) The remove-arm code (Jamadandi ProxyDelete) was verified to exist and be usable (RelationalML/SpectralPruningBraess; pure NetworkX/SciPy, CPU-friendly, runs on Cora node classification), upgrading the remove-arm from a random fallback to a principled spectral method. (3) The DeepRobust attack–defense pipeline was installed (working around a broken gensim pin via -no-deps plus prebuilt PyG extension wheels) and validated end-to-end on CPU; Table 3 reports the smoke-test numbers.

To do. Build the core pipeline and headline table over {clean, ptb 0.05, ptb 0.10} (Task 2); adapt FoSR to node tasks (Task 3); implement the scoped Jacobian probe (Task 4); add Citeseer and integrate ProxyDelete (Task 5); prepare slides and the progress summary (Task 6).

Setting (Cora, Metattack pth = 0.05)	Clean	Robust (poisoned)	Gap
GCN (Planetoid public split, 5 seeds)	80.30 $\pm$ 0.47	—	—
GCN (DeepRobust attack split)	84.05	77.72	6.34
GCN-Jaccard (DeepRobust attack split)	—	79.38	—

Table 3: Preliminary numbers (%). Top row: the clean baseline on the standard Planetoid split. Bottom rows: the Task 1 end-to-end pipeline smoke test on the attack split used by robustness papers (single seed, untuned) — enough to confirm the path works; the full tuned grid is Task 2.

Obstacles and concerns. (i) DeepRobust (0.2.9, 2023) is brittle against current PyTorch/NumPy — resolved by a clean dependency install and by running on CPU for these small graphs, sidestepping Blackwell-GPU (sm_120) issues. (ii) FoSR’s reference code targets graph classification and must be adapted to node tasks. (iii) The Jacobian probe is the hardest component and is scoped small first. (iv) The timeline is tight, so Cora is prioritized and all other datasets and the adaptive-attack setting are explicitly stretch goals.

Reproducibility. All numbers in this report are produced by fixed-seed scripts (the GCN/Cora baseline and the DeepRobust attack–defense pipeline spike) and are reproducible end-to-end; we report only real, measured results, and the codebase is organized so that each table can be regenerated from a single command.

References

- [1] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations (ICLR)*, 2021.
- [2] Francesco Di Giovanni, Lorenzo Giusti, Federico Barbero, Giulia Luise, Pietro Lio, and Michael M. Bronstein. On over-squashing in message passing neural networks: The impact of width, depth, and topology. In *International Conference on Machine Learning (ICML)*, 2023.
- [3] Adarsh Jamadandi, Celia Rubio-Madrigal, and Rebekka Burkholz. Spectral graph pruning against over-squashing and over-smoothing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [4] Wei Jin, Yao Ma, Xiaorui Liu, Xianfeng Tang, Suhang Wang, and Jiliang Tang. Graph structure learning for robust graph neural networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020.
- [5] Kedar Karhadkar, Pradeep Kr. Banerjee, and Guido Montúfar. FoSR: First-order spectral rewiring for addressing oversquashing in GNNs. In *International Conference on Learning Representations (ICLR)*, 2023.
- [6] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [7] Yaxin Li, Wei Jin, Han Xu, and Jiliang Tang. DeepRobust: A PyTorch library for adversarial attacks and defenses. *arXiv preprint arXiv:2005.06149*, 2020.
- [8] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations (ICLR)*, 2022.
- [9] Huijun Wu, Chen Wang, Yuriy Tyshetskiy, Andrew Docherty, Kai Lu, and Liming Zhu. Adversarial examples for graph data: Deep insights into attack and defense. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.

- [10] Daniel Zügner and Stephan Günnemann. Adversarial attacks on graph neural networks via meta learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- [11] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. Adversarial attacks on neural networks for graph data. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2018.